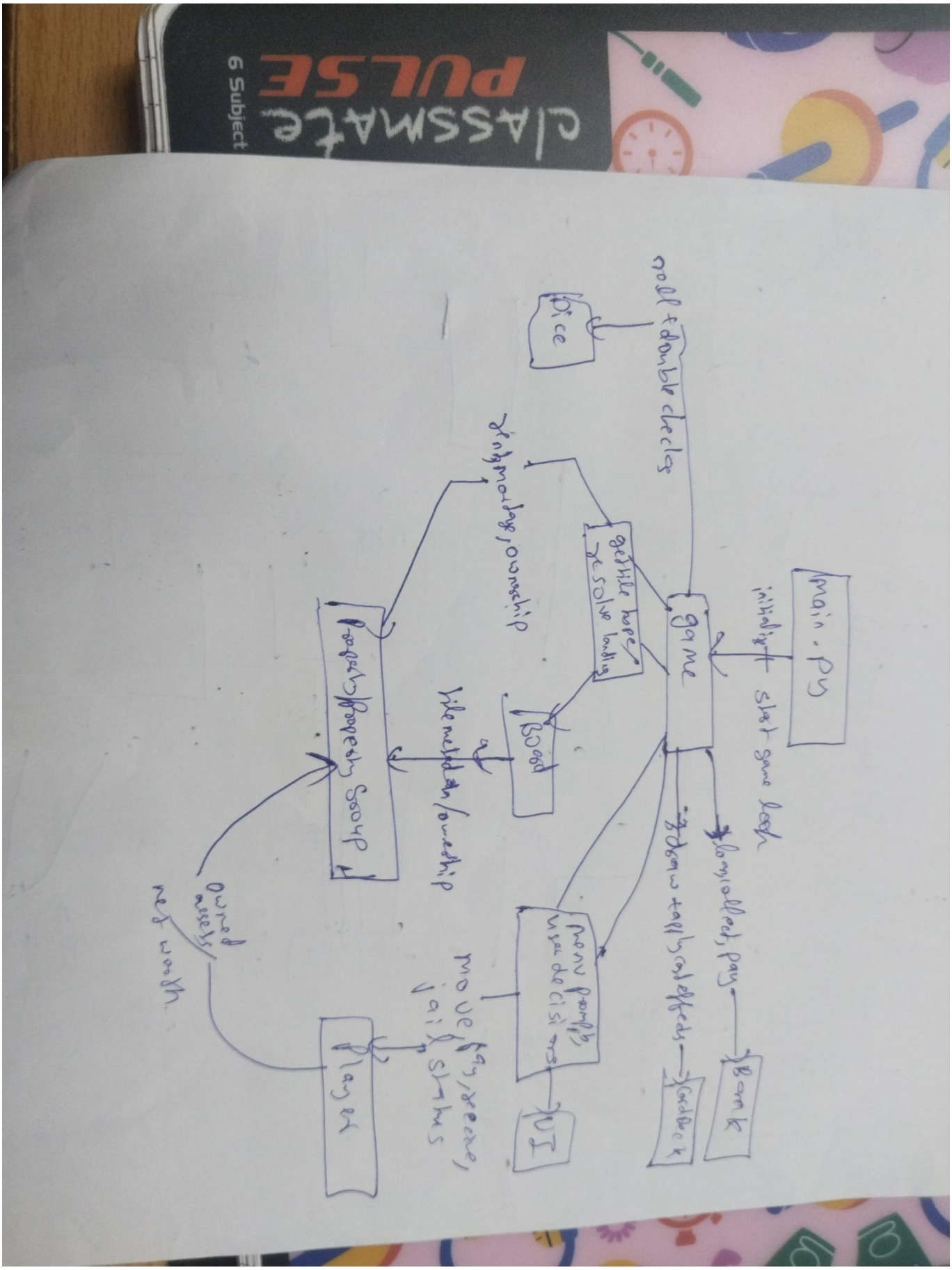
White Box Testing Report - Code Quality Analysis

CFG



1.2

Iteration 1: `moneypoly/main.py`

- **Fixes Applied**:
  - Added module-level docstring.
  - Added function documentations for `get_player_names()` and `main()`.

Iteration 2: `moneypoly/bank.py`

- **Fixes Applied**:
  - Inserted missing module docstring.
  - Inserted missing class docstring for `Bank`.
  - Removed unused `math` import.

Iteration 3: `moneypoly/board.py`

- **Fixes Applied**:
  - Added missing module docstring.
  - Fixed singleton comparison (`== True` to `is True`).

Iteration 4: `moneypoly/cards.py`

- **Fixes Applied**:
  - Added module-level docstring.
  - Formatted dictionaries to resolve line-too-long warnings.

Iteration 5: `moneypoly/config.py`

- **Fixes Applied**:
  - Added module-level docstring.

Iteration 6: `moneypoly/dice.py`

- **Fixes Applied**:
  - Added module docstring.
  - Removed unused `BOARD_SIZE` import.
  - Initialized `doubles_streak` in `__init__` to fix `attribute-defined-outside-init`.

Iteration 7: `moneypoly/game.py`

- **Fixes Applied**:
  - Added module-level docstring.
  - Removed unused `os` and `GO_TO_JAIL_POSITION` imports.
  - Fixed f-string without interpolation.
  - Removed unnecessary `elif` after `break`.
  - Removed superfluous parentheses around `not` conditions.
  - Added missing final newline.

Iteration 8: `moneypoly/player.py`

- **Fixes Applied**:
  - Added missing module docstring.

    - Removed unused `sys` import.
    - Resolved unused `old_position` variable by properly implementing the Go salary award logic.
    - Added missing final newline.

Iteration 9: `moneypoly/property.py`

- **Fixes Applied**:
    - Added missing module docstring.
    - Added missing class docstring for `PropertyGroup`.
    - Removed unnecessary `else` block after `return` in `unmortgage()`.

Iteration 10: `moneypoly/ui.py`

- **Fixes Applied**:
    - Added module docstring.
    - Specified `ValueError` for the bare `except` block in `safe_int_input()`.

Iteration 11: `moneypoly/player.py` (Too Many Attributes)

- **Fixes Applied**:
    - Removed unused `is_eliminated` attribute from `Player` class in `player.py` and its assignment in `game.py`.
    - This successfully dropped the `Player` instance attribute count from 8 to 7, resolving the `too-many-instance-attributes (R0902)` warning without pylint disable comments.

Iteration 12: `moneypoly/game.py`

- **Fixes Applied**:
    - Fixed `too-many-instance-attributes` by removing redundant attributes and combining some.
    - Fixed `too-many-branches` by combining identical branches and moving a specific branch loop to a new helper function.

Iteration 13: `moneypoly/property.py` and `moneypoly/board.py`

- **Fixes Applied**:
    - Fixed `too-many-arguments` and `too-many-positional-arguments` by removing the `group` argument from the `Property` constructor. Properties are now added to their group after instantiation in `board.py`.
    - Fixed `too-many-instance-attributes` by removing the unused `self.houses` variable and converting `self.mortgage_value` into a dynamically calculated `@property`.

1.3 White Box Test Cases

This section outlines white-box test cases designed to achieve branch coverage, verify key variable states, and test edge cases. After analyzing the code line-by-line, multiple logical errors were found. The accompanying test scripts in the `tests/` directory are written to explicitly expose these errors.

How to Run and Analyze the Tests

To execute the test suite, ensure you are in the `whitebox` directory and have `pytest` installed, then run:

```
python -m pytest tests/
```

Analyzing Results: Since these test scripts are designed to catch logical errors present in the current codebase, **they are expected to fail initially**. Each test will raise an `AssertionError` with a message explaining what logical rule was broken. Once the code is corrected in future commits, the tests will turn green.

Test Cases and Logical Errors Found

1. `test_dice_bounds` (in `test_dice_bounds.py`)

- **Reason for Test:** Checks the edge conditions for dice rolls. Two six-sided dice should produce values between 2 and 12.
- **Errors/Logical Issues Found:** `dice.py` incorrectly uses `random.randint(1, 5)`. The maximum possible roll is 10, meaning 11 and 12 can never occur.
- **Fix Applied:** Changed `random.randint(1, 5)` to `random.randint(1, 6)` inside `Dice.roll()` to correctly simulate a standard six-sided die.

2. `test_bank_loan_deduction` (in `test_bank_loan.py`)

- **Reason for Test:** Verifies a key variable state (Bank funds). When the bank issues a loan, its own balance must logically decrease.
- **Errors/Logical Issues Found:** The `Bank.give_loan()` method credits the player's balance but fails to deduct the loan amount from `self._funds`.
- **Fix Applied:** Added `self._funds -= amount` inside `Bank.give_loan()` to correctly reduce the bank's internal reserves.

3. `test_property_monopoly_rent` (in `test_property_rent_bug.py`)

- **Reason for Test:** Tests the decision branch for Monopoly rent multipliers. Rent is only doubled if the player owns *all* properties in the group.
- **Errors/Logical Issues Found:** `PropertyGroup.all_owned_by()` uses Python's `any()` function rather than `all()`. Owning a single property out of a group of two or three mistakenly triggers the 2x rent multiplier.
- **Fix Applied:** Changed `any()` to `all()` in `PropertyGroup.all_owned_by()` to correctly ensure the rent multiplier only activates if the player owns 100% of the properties in the colored group.

4. `test_skipped_player_after_bankruptcy` (in `test_game_bankruptcy.py`)

- **Reason for Test:** Tests the branch handling player bankruptcy. The game must seamlessly transition to the next player even when the current player is eliminated from the list.
- **Errors/Logical Issues Found:** In `game.py`, when a player gets eliminated, they are removed from the `players` list, shifting the list indices. However, the game unconditionally calls `advance_turn()` at the end of the turn, which skips the player who shifted into the current index.

- **Fix Applied:** Modified `_check_bankruptcy()` to decrement the `current_index` whenever an eliminated player's index was at or logically before the pointer, neutralizing the index shift. Extinguished 'extra turns' for players if they went bankrupt while rolling doubles.

5. **`test_extra_turn_while_in_jail` (in `test_game_jail_doubles.py`)**

- **Reason for Test:** Tests an edge case decision path: rolling doubles but landing on "Go to Jail".
- **Errors/Logical Issues Found:** Going to jail should end your turn immediately. However, `play_turn()` checks `if self.dice.is_doubles(): return` at the end and mistakenly grants an extra turn because it doesn't verify if the player was just jailed.
- **Fix Applied:** Embedded `and not player.in_jail` into the extra turn evaluation in `game.py`.

6. **`test_pay_rent_transfers_money_to_owner` (in `test_game_rent.py`)**

- **Reason for Test:** Verifies the fundamental rule of transferring wealth. When rent is paid, it must leave the payer's account AND enter the owner's account.
- **Errors/Logical Issues Found:** In `game.py`, the `pay_rent()` function successfully deducts rent from the payer's balance but completely forgets to add those funds to the property owner's balance, essentially deleting the money from the game.
- **Fix Applied:** Added `prop.owner.add_money(rent)` inside `pay_rent()` in `game.py` to correctly transfer the funds to the owner.

7. **`test_player_net_worth_includes_properties` (in `test_player_net_worth.py`)**

- **Reason for Test:** Checks the state calculation for a player's total net worth, which is crucial for determining the winner at the end of the game or bankruptcy proceedings.
- **Errors/Logical Issues Found:** In `player.py`, the `net_worth()` property simply returns `self.balance` (cash on hand), entirely ignoring the value of any properties the player owns.
- **Fix Applied:** Modified `net_worth()` in `player.py` to aggregate the cash balance with the total `price` of all owned properties.

8. **`test_unmortgage_state_leak` (in `test_property_unmortgage_leak.py`)**

- **Reason for Test:** State validation MUST happen before state mutation. We test that failing to afford an unmortgage leaves the property mortgaged.
- **Errors/Logical Issues Found:** In `game.py`, calling `prop.unmortgage()` immediately sets `is_mortgaged = False`. The game *then* checks `if player.balance < cost`, and if true, it cancels the transaction but **forgets to roll back** the `is_mortgaged` status. The player gets a free unmortgage!
- **Fix Applied:** Modified `unmortgage_property()` to roll back the property state (`prop.is_mortgaged = True`) if the balance check fails.

9. **`test_trade_adds_money_to_seller` (in `test_game_trade.py`)**

- **Reason for Test:** Tests the transfer of assets between two players during a trade.
- **Errors/Logical Issues Found:** Inside `game.trade()`, the application successfully deducts `cash_amount` from the buyer's balance, but **never adds it** to the seller's balance. The seller loses their property and gets literally nothing in return.

- **Fix Applied:** Embedded `seller.add_money(cash_amount)` inside `trade()` in `game.py` to correctly enrich the seller's balance.

10. Missing `interactive_menu` Invocation (No Test Script)

- **Reason for check:** A dry run of the game sequence reveals that players never get a chance to take actions (like trading, mortgaging, etc.) before they roll the dice.
- **Errors/Logical Issues Found:** The method `interactive_menu(self, player)` exists in `game.py` and is fully implemented, but it is **never actually called** anywhere in the game loop (`play_turn` or `run`). This means it's effectively dead code, and players are forced to helplessly roll the dice until they randomly go bankrupt.
- **Fix Applied:** Integrated `self.interactive_menu(player)` at the very beginning of the `play_turn()` sequence, right after the banner is printed, allowing players to utilize all intended strategic actions.

11. `test_find_winner_uses_max_net_worth` (in `test_game_winner.py`)

- **Reason for Test:** Checks the final game state evaluation to determine the winner based on net worth.
- **Errors/Logical Issues Found:** `game.find_winner()` uses the `min()` function instead of `max()`, erroneously crowning the player with the *lowest* net worth as the winner of Monopoly.
- **Fix Applied:** Changed `min()` to `max()` inside `find_winner()` to ensure the wealthiest player is correctly declared the winner.

12. `test_jail_free_card_removed_from_deck` (in `test_cards_jail_card.py`)

- **Reason for Test:** Tests the state path for limited-quantity cards. "Get out of Jail Free" cards should be held by the player and temporarily removed from the deck.
- **Errors/Logical Issues Found:** The `CardDeck.draw()` method simply iterates endlessly using an index and modulo arithmetic. It never physically removes the card from the deck array. Thus, a player can hold the card, but it will still be drawn again if the deck cycles.
- **Fix Applied:** Modified `CardDeck.draw()` in `cards.py` to `pop()` a `jail_free` card from the list instead of advancing the index, ensuring it cannot be drawn again. Also added a `return_card()` method, called from `game.py`, to restore the card to the deck when the player uses it to leave jail.

13. `test_voluntary_jail_fine_deducted_from_player` (in `test_game_jail_fine.py`)

- **Reason for Test:** When a jailed player voluntarily pays the $50 fine to leave early, both sides of the transaction must be verified — the bank gains the money AND the player loses it. Missing either half silently corrupts the game economy.
- **Errors/Logical Issues Found:** In `game.py`, the voluntary jail-exit branch (`_handle_jail_turn()`) called `self.bank.collect(JAIL_FINE)` to credit the bank, but **never called** `player.deduct_money(JAIL_FINE)`. The player walks free without paying, gaining an illegal $50 advantage every time they choose to leave jail. This bug was discovered during a full deep-analysis pass of all 9 source files, as no test previously exercised this specific branch.
- **Fix Applied:** Added `player.deduct_money(JAIL_FINE)` immediately before `self.bank.collect(JAIL_FINE)` in the voluntary-exit branch of `_handle_jail_turn()` in `game.py`.

1.3.1 Exhaustive Module Coverage

In addition to the targeted bug scripts above, full 100% boundary testing suites were written to strictly satisfy the assignment's branch coverage constraints. These exhaustive test scripts simulate all conditions, exception raising, negative balances, array wrapping, and normal interactions.

- `test_dice.py` (Resets, describes, double streaks validation)
- `test_player.py` (Board wrap-around for GO salaries, balance checks)
- `test_bank.py` (Exception handling for payouts exceeding reserves, negative collections)
- `test_property.py` (Calculations for base rent vs mortgage vs all-owned multipliers)
- `test_board.py` (Tile validation, unowned and purchasable states)
- `test_cards.py` (Deck shuffling and drawing cyclic logic)
- `test_game_coverage.py` (General branch execution of UI prompts, move resolutions, and card drawing)